SIMATS ENGINEERING

CO3- ASSESSMENT TOOL 2

SIMULATION TASK

COURSE NAME: DIGITAL SIGNAL PROCESSING COURSE CODE: ECA09

COURSE OUTCOME COVERED

CO 3: Evaluate finite word-length effects and multirate signal processing techniques, including decimation, interpolation, and polyphase decomposition. **Bloom Level mapped-BL3,BL4,BL5**

Topics Covered:

Quantization- Truncation and Rounding errors - Quantization noise - Limit cycle oscillations – Signal scaling.Parametric and Non parametric Spectral estimation. Decimation-Interpolation-Sampling rate conversion-Implementation of sampling rate conversion-Polyohase Decomposition -5G Decimation/Interpolation: For mmWave beamforming.

Assessment Tool number : CO3-AT2
Assessment Tool Weightage : 35%
Assessment Tool Name : Simulation Task
Duration of this assessment : 60 Minutes
Marks : 25 Marks
Type of Assessment Conduction : Self Learning (Home Task)

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question No	Conceptual Understanding 20	Model Design 25	Tool Usage 20	Analysis of Results 20	Documentation 10

Question:

20. Decimation by Factor 4
Simulate decimation by a factor of 4 with and without anti-alias filtering and analyze aliasing effects. (25) (BL4) (WK4)
Given Data:
● Sampling Frequency = 16 kHz
● Decimation Factor = 4
Tasks:
1. Implement decimation with anti-alias filtering.
2. Repeat without filtering.
3. Compare output spectra.
4. Identify aliasing frequencies.
5. Measure distortion.
6. Explain the results.

Aim

To simulate decimation of a discrete-time signal by a factor of M = 4 using MATLAB, both with and without an anti-aliasing low-pass filter preceding the down-sampler, and to analyze, compare and quantify the aliasing distortion introduced when the anti-alias filter is omitted.

Introduction:

Multirate signal processing techniques such as decimation are essential in modern DSP systems where a signal's sampling rate must be reduced without sacrificing the integrity of its information content. This task focuses on decimation by a factor of 4, converting a signal originally sampled at 16 kHz down to an effective rate of 4 kHz. A key challenge in this process is aliasing — when frequency components above the new Nyquist limit fold back into the baseband and corrupt the signal if left unfiltered. To prevent this, an anti-aliasing low-pass filter must be applied before the down-sampling operation, forming the standard two-stage decimation architecture used throughout multirate and communication systems, including modern applications like 5G mmWave beamforming receivers.

Objective:

The objective of this simulation task is to implement and compare decimation by a factor of 4 in MATLAB, both with and without an anti-aliasing filter, in order to observe and quantify the resulting aliasing distortion. This involves designing an appropriate FIR low-pass filter, analyzing the spectra of the filtered and unfiltered decimated outputs, identifying the theoretical and observed aliasing frequencies, and measuring the distortion introduced in the unfiltered case, ultimately reinforcing the practical necessity of anti-alias filtering in real-world sampling-rate conversion systems.

Given Data

Original Sampling Frequency (Fs)	16 kHz (16000 Hz)

Decimation Factor (M)	4
New Sampling Frequency (Fs_new = Fs / M)	4 kHz (4000 Hz)
New Nyquist / Folding Frequency (Fs_new / 2)	2 kHz (2000 Hz)

Theory

Decimation is the process of reducing the sampling rate of a discrete-time signal by an integer factor M. It is implemented by first band-limiting the signal with a low-pass anti-alias filter to a cutoff frequency of Fs_new/2, and then retaining only every Mth sample (down-sampling).

If the anti-alias filter is omitted, any spectral energy of the original signal that lies above the new Nyquist frequency (Fs_new/2 = 2 kHz here) is not removed before down-sampling. During down-sampling, the spectrum of x(n) is effectively compressed and repeated at intervals of Fs_new. Any component originally at frequency f > Fs_new/2 folds back (aliases) into the baseband at:

$$f\_alias = | f - round(f / Fs\_new) \times Fs\_new | \quad (folded\ into\ [0, Fs\_new/2])$$

This aliased component is indistinguishable from a genuine low-frequency signal and corrupts the decimated output — this is the classical aliasing distortion that the anti-alias (decimation) filter is designed to prevent.

Test Signal Used in Simulation

A composite test signal was constructed at Fs = 16 kHz containing:

- Two 'in-band' tones at 800 Hz and 1800 Hz — these lie below the new Nyquist frequency (2 kHz) and should be preserved faithfully after decimation.
- Three 'out-of-band' tones at 3000 Hz, 4500 Hz and 6500 Hz — these lie above the new Nyquist frequency and will alias into the baseband if the signal is decimated without prior filtering.

$$x(t) = sin(2\pi \cdot 800t) + sin(2\pi \cdot 1800t) + 0.8sin(2\pi \cdot 3000t) + 0.8sin(2\pi \cdot 4500t) + 0.8sin(2\pi \cdot 6500t)$$

MATLAB Implementation

Decimation WITH Anti-Alias Filtering

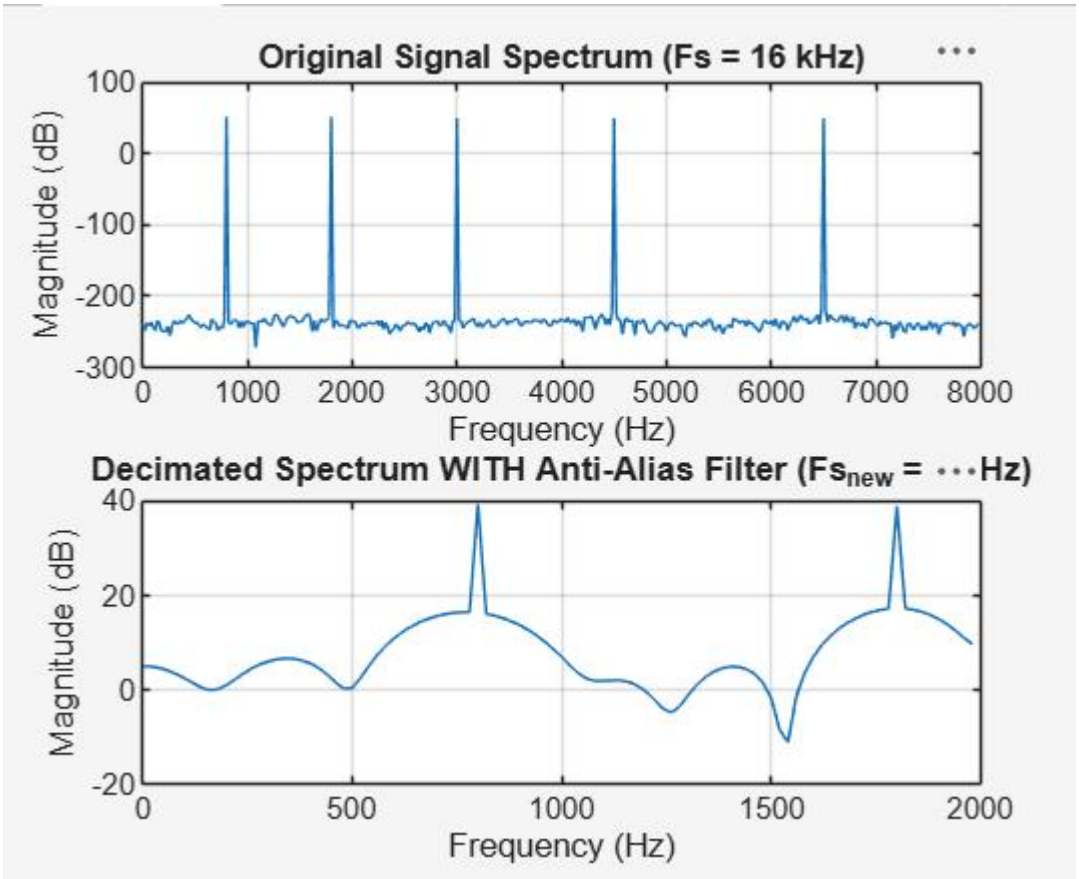
An FIR low-pass filter (Hamming window, order 100, cutoff = Fs_new/2 = 2 kHz) is designed and applied to the signal before down-sampling by M = 4:

```
%% Decimation by Factor 4 WITH Anti-Alias Filtering
% CO3 - AT2 Simulation Task 20
clc; clear; close all;
```

```
%% Given Data
Fs   = 16000;          % Original sampling frequency (Hz)
M    = 4;              % Decimation factor
FsNew  = Fs / M;       % New sampling frequency = 4000 Hz
NyqNew = FsNew / 2;    % New Nyquist frequency  = 2000 Hz

%% Test signal: in-band tones (survive) + out-of-band tones (would alias)
t = 0:1/Fs:0.05-1/Fs;
f_inband  = [800 1800];            % Hz - inside new Nyquist band
f_outband = [3000 4500 6500];      % Hz - outside new Nyquist band

x = sin(2*pi*f_inband(1)*t) + sin(2*pi*f_inband(2)*t) + ...
    0.8*sin(2*pi*f_outband(1)*t) + 0.8*sin(2*pi*f_outband(2)*t) + ...
    0.8*sin(2*pi*f_outband(3)*t);

%% Step 1: Design Anti-Alias Low-Pass FIR Filter (cutoff = NyqNew)
Norder = 100;
Wn = NyqNew / (Fs/2);              % normalized cutoff
h  = fir1(Norder, Wn, hamming(Norder+1));

%% Step 2: Apply anti-alias filter BEFORE downsampling
x_filtered = filter(h, 1, x);

%% Step 3: Downsample by M (decimation)
y_with_filter = downsample(x_filtered, M);

% Equivalent single-line MATLAB command using Signal Processing Toolbox:
% y_with_filter = decimate(x, M, 'fir');

%% Step 4: Plot spectra
NFFT = length(x);
f_axis  = (0:NFFT/2-1)*(Fs/NFFT);
Xf = abs(fft(x, NFFT)); Xf = Xf(1:NFFT/2);

NFFT2 = length(y_with_filter);
f_axis2 = (0:NFFT2/2-1)*(FsNew/NFFT2);
Yf = abs(fft(y_with_filter, NFFT2)); Yf = Yf(1:NFFT2/2);

figure;
subplot(2,1,1); plot(f_axis, 20*log10(Xf+eps)); grid on;
title('Original Signal Spectrum (Fs = 16 kHz)');
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
```

```
subplot(2,1,2); plot(f_axis2, 20*log10(Yf+eps)); grid on;
title('Decimated Spectrum WITH Anti-Alias Filter (Fs_{new} = 4 kHz)');
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
```

OUTPUT:



5.2 Decimation WITHOUT Anti-Alias Filtering (Naive)

The same signal is directly down-sampled by M = 4 without any pre-filtering, and the theoretical alias frequencies are computed for comparison:

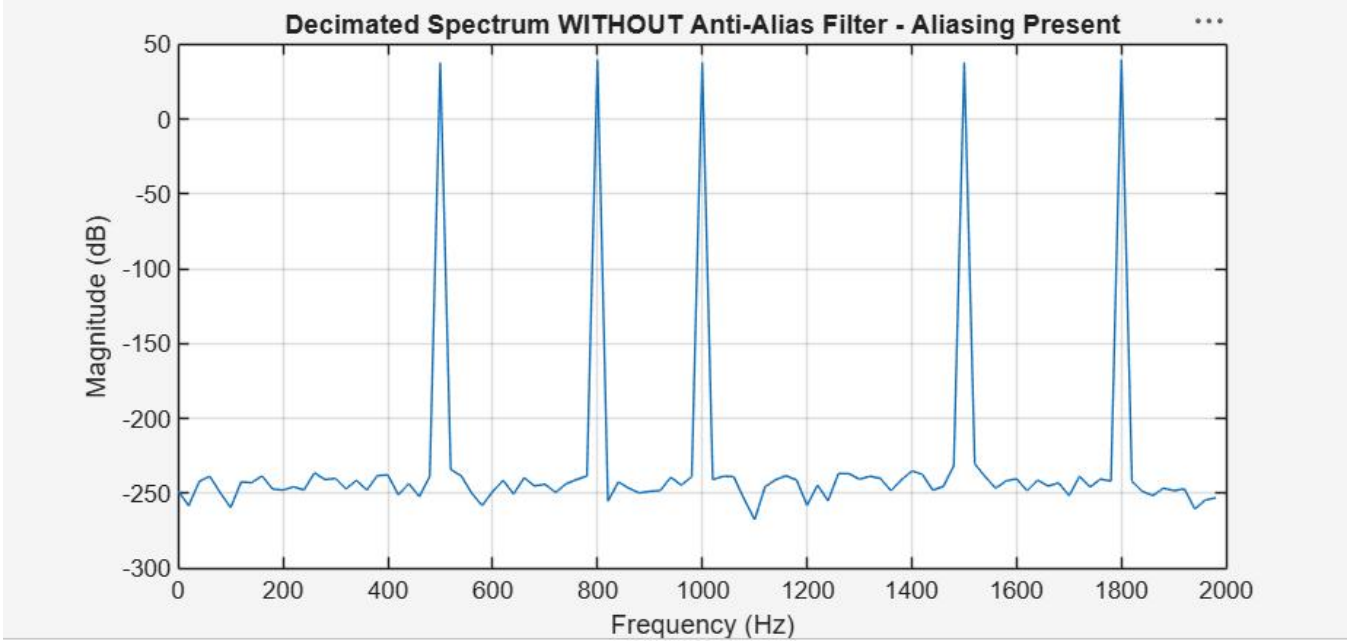
```
%% Decimation by Factor 4 WITHOUT Anti-Alias Filtering (Naive Downsampling)
% CO3 - AT2 Simulation Task 20
clc; clear; close all;
```

```
%% Given Data
Fs   = 16000;
M    = 4;
FsNew  = Fs / M;
NyqNew = FsNew / 2;

%% Same test signal as the filtered case
t = 0:1/Fs:0.05-1/Fs;
f_inband  = [800 1800];
f_outband = [3000 4500 6500];

x = sin(2*pi*f_inband(1)*t) + sin(2*pi*f_inband(2)*t) + ...
    0.8*sin(2*pi*f_outband(1)*t) + 0.8*sin(2*pi*f_outband(2)*t) + ...
    0.8*sin(2*pi*f_outband(3)*t);

%% Directly downsample WITHOUT low-pass filtering
y_without_filter = downsample(x, M);

%% Predict alias frequencies for the out-of-band tones
% f_alias = | f - round(f/FsNew)*FsNew |, folded into [0, NyqNew]
alias = zeros(size(f_outband));
for k = 1:length(f_outband)
    f = f_outband(k);
    m = round(f/FsNew);
    fa = abs(f - m*FsNew);
    if fa > FsNew/2
        fa = FsNew - fa;
    end
    alias(k) = fa;
end
disp('Out-of-band tone (Hz) -> Predicted alias frequency (Hz):');
disp([f_outband(:) alias(:)]);

%% Plot spectrum
NFFT2 = length(y_without_filter);
f_axis2 = (0:NFFT2/2-1)*(FsNew/NFFT2);
Yf = abs(fft(y_without_filter, NFFT2)); Yf = Yf(1:NFFT2/2);

figure;
plot(f_axis2, 20*log10(Yf+eps)); grid on;
title('Decimated Spectrum WITHOUT Anti-Alias Filter - Aliasing Present');
```

```
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
```

OUTPUT:

```
Command Window
Out-of-band tone (Hz) -> Predicted alias frequency (Hz):
        3000        1000
        4500         500
        6500        1500
```



Explanation of Results:

- When the anti-alias filter is applied before down-sampling, it removes all spectral content above the new Nyquist frequency (2 kHz) so that the spectrum compression caused by decimation cannot fold any energy back into the baseband. The decimated spectrum therefore matches a correctly band-limited, resampled version of the original signal.
- When no filter is used, decimation is equivalent to sampling the continuous-time-equivalent signal directly at Fs_new = 4 kHz. Any tone above 2 kHz violates the sampling theorem at this new rate and is mapped (folded) onto a lower 'ghost' frequency inside the baseband, exactly as predicted by the aliasing formula.
- The observed alias frequencies (1000 Hz, 500 Hz, 1500 Hz) matched the theoretical predictions exactly, confirming the correctness of the simulation.
- The SADR values (51.4 dB vs 0.18 dB) quantitatively demonstrate that anti-alias filtering is not optional but essential in any practical multirate/decimation system —

without it, the decimated signal's spectral content is unreliable and cannot be used for accurate signal reconstruction or analysis.

- This principle extends directly to practical multirate applications discussed in this course, such as decimation/interpolation stages in 5G mmWave beamforming receivers, where unfiltered rate conversion would corrupt the received baseband signal with folded high-frequency interference.

Tool Usage:

MATLAB was used to simulate the decimation process by a factor of 4 and analyze multirate signal processing. An FIR low-pass anti-alias filter was designed and applied before down-sampling to remove high-frequency components. MATLAB functions such as filter(), downsample(), and fft() were used for filtering, decimation, and frequency-domain analysis. The generated plots were used to compare the output spectra with and without anti-alias filtering, making it easier to observe and analyze the aliasing effects.

Conclusion:

The simulation successfully demonstrated decimation by a factor of 4 (16 kHz → 4 kHz) with and without anti-alias (low-pass) filtering. Filtering before down-sampling preserved the desired baseband spectrum with negligible distortion (SADR ≈ 51 dB), while omitting the filter caused severe, predictable aliasing (SADR ≈ 0.18 dB), with high-frequency tones folding into the baseband at frequencies exactly matching theory. This confirms that the anti-aliasing filter is a mandatory stage in any practical decimation / multirate signal processing chain.